jart / cosmopolitan

Type / to search

Code Issues 140 Pull requests 14 Discussions Actions Wiki Security Insights

cosmopolitan Public

9 Branches 103 Tags Go to file t Go to file Add file Code

jart Upgrade to superconfigure z0.0.51 ff1a0d1 · last week 2,469 Commits

.github	Remove testing label from labele...	2 months ago
ape	Remove .internal from more hea...	last week
build	Freshen build/bootstrap/cocmd	2 weeks ago
ctl	Release Cosmopolitan v3.6.0	3 weeks ago
dsp	Remove .internal from more hea...	last week
examples	Remove .internal from more hea...	last week
libc	Remove .internal from more hea...	last week
net	Remove .internal from more hea...	last week
test	Remove .internal from more hea...	last week
third_party	Remove .internal from more hea...	last week
tool	Upgrade to superconfigure z0.0....	last week
usr/share	Implement proper time zone su...	3 months ago
.clang-format	Update .clang-format	4 months ago
.git-blame-ignore-revs	Add last commit to .git-blame-ig...	last week
.gitattributes	Make improvements	10 months ago
.gitignore	Release Cosmopolitan v3.2.4	7 months ago
CONTRIBUTING.md	Fix broken link	6 months ago
LICENSE	Add title to the LICENSE file	3 years ago
Makefile	Avoid legacy instruction penaltie...	2 weeks ago
README.md	Restore support for AMD K8	3 weeks ago

About

build-once run-anywhere c library

#windows #linux #freebsd #openbsd #containers #zip #darwin #polyglot #libc #netbsd #efi #bios

Readme
ISC license
Activity
17.6k stars
166 watching
603 forks
Report repository

Releases 50

Cosmopolitan v3.6.2 Latest
2 weeks ago

+ 49 releases

Sponsor this project

jart Justine Tunney

Sponsor

Learn more about GitHub Sponsors

Packages

No packages published

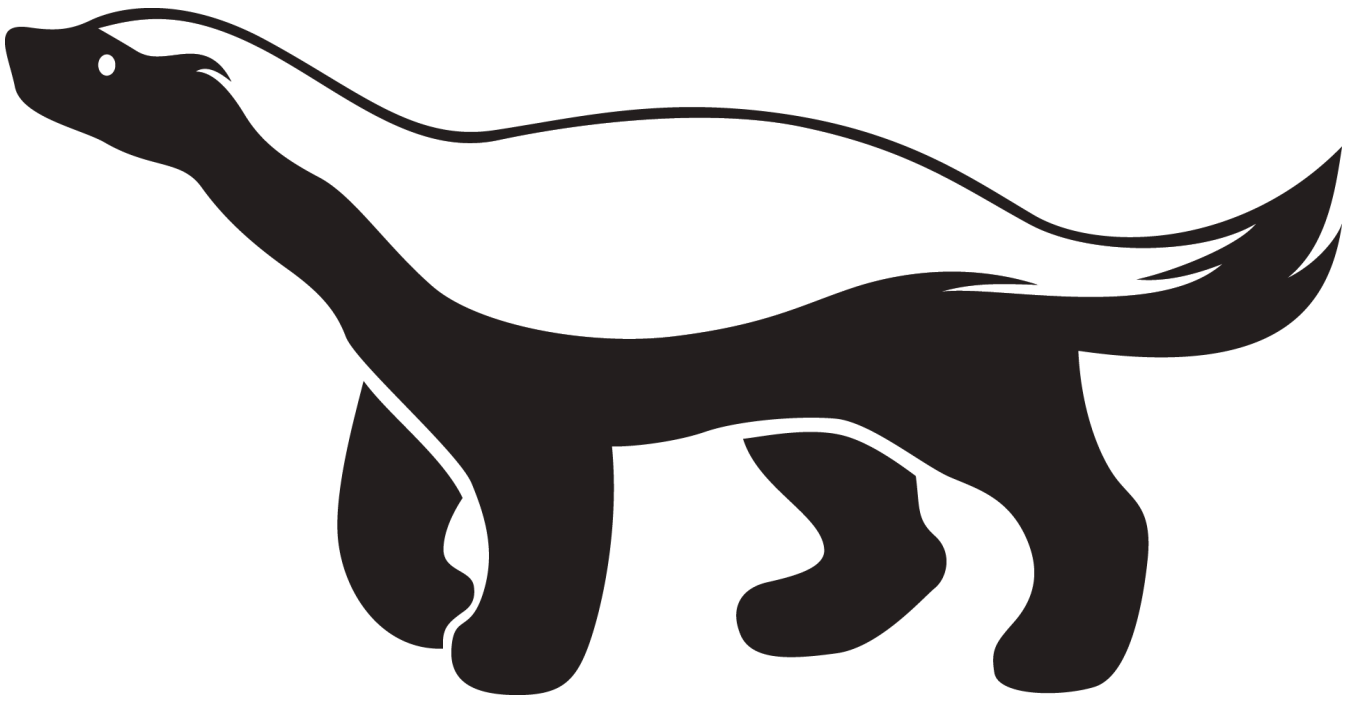
Contributors 77

+ 63 contributors

Languages

C 61.0% POV-Ray SDL 24.1% Assembly 5.2% Lua 4.1% C++ 1.9% Shell 1.7% Other 2.0%

README ISC license



build passing

Cosmopolitan

Cosmopolitan Libc makes C a build-once run-anywhere language, like Java, except it doesn't need an interpreter or virtual machine. Instead, it reconfigures stock GCC and Clang to output a POSIX-approved polyglot format that runs natively on Linux + Mac + Windows + FreeBSD + OpenBSD + NetBSD + BIOS with the best possible performance and the tiniest footprint imaginable.

Background

For an introduction to this project, please read the actually portable executable blog post and cosmopolitan libc website. We also have API documentation.

Getting Started

You can start by obtaining a release of our `cosmocc` compiler from https://cosmo.zip/pub/cosmocc/.

```
mkdir -p cosmocc
cd cosmocc
wget https://cosmo.zip/pub/cosmocc/cosmocc.zip
unzip cosmocc.zip
```

Here's an example program we can write:

```
// hello.c
#include <stdio.h>

int main() {
  printf("hello world\n");
}
```

It can be compiled as follows:

```
cosmocc -o hello hello.c
./hello
```

The Cosmopolitan Libc runtime links some heavyweight troubleshooting features by default, which are very useful for developers and admins. Here's how you can log system calls:

```
./hello --strace
```

Here's how you can get a much more verbose log of function calls:

```
./hello --ftrace
```

You can use the Cosmopolitan's toolchain to build conventional open source projects which use autotools. This strategy normally works:

```
export CC=x86_64-unknown-cosmo-cc
export CXX=x86_64-unknown-cosmo-c++
./configure --prefix=/opt/cosmos/x86_64
make -j
make install
```

Cosmopolitan Source Builds

Cosmopolitan can be compiled from source on any of our supported platforms. The Makefile will download cosmocc automatically.

It's recommended that you install a systemwide APE Loader. This command requires `sudo` access to copy the `ape` command to a system folder and register with binfmt_misc on Linux, for even more performance.

```
ape/apeinstall.sh
```

You can now build the mono repo with any modern version of GNU Make. To make life easier, we've included one in the cosmocc toolchain, which is guaranteed to be compatible and furthermore includes our extensions for doing build system sandboxing.

```
build/bootstrap/make -j8
o//examples/hello
```

Since the Cosmopolitan repository is very large, you might only want to build one particular thing. Here's an example of a target that can be compiled relatively quickly, which is a simple POSIX test that only depends on core LIBC packages.

```
rm -rf o//libc o//test
build/bootstrap/make o//test/posix/signal_test
o//test/posix/signal_test
```

Sometimes it's desirable to build a subset of targets, without having to list out each individual one. For example if you wanted to build and run all the unit tests in the `TEST_POSIX` package, you could say:

```
build/bootstrap/make o//test/posix
```

Cosmopolitan provides a variety of build modes. For example, if you want really tiny binaries (as small as 12kb in size) then you'd say:

```
build/bootstrap/make m=tiny
```

You can furthermore cut out the bloat of other operating systems, and have Cosmopolitan become much more similar to Musl Libc.

```
build/bootstrap/make m=tinylinux
```

For further details, see //build/config.mk.

Debugging

To print a log of system calls to stderr:

```
cosmocc -o hello hello.c
./hello --strace
```

To print a log of function calls to stderr:

```
cosmocc -o hello hello.c
./hello --ftrace
```

Both strace and ftrace use the unbreakable kprintf() facility, which is able to be sent to a file by setting an environment variable.

```
export KPRINTF_LOG=log
./hello --strace
```

GDB

Here's the recommended `~/.gdbinit` config:

```
set host-charset UTF-8
set target-charset UTF-8
set target-wide-charset UTF-8
set osabi none
set complaints 0
set confirm off
set history save on
set history filename ~/.gdb_history
define asm
  layout asm
  layout reg
end
define src
  layout src
  layout reg
end
src
```

You normally run the `.dbg` file under gdb. If you need to debug the `` file itself, then you can load the debug symbols independently as

```
gdb foo -ex 'add-symbol-file foo.dbg 0x401000'
```

Platform Notes

Shells

If you use zsh and have trouble running APE programs try `sh -c ./prog` or simply upgrade to zsh 5.9+ (since we patched it two years ago). The same is the case for Python `subprocess`, old versions of fish, etc.

Linux

Some Linux systems are configured to launch MZ executables under WINE. Other distros configure their stock installs so that APE programs will print "run-detectors: unable to find an interpreter". For example:

```
jart@ubuntu:~$ wget https://cosmo.zip/pub/cosmos/bin/dash
jart@ubuntu:~$ chmod +x dash
jart@ubuntu:~$ ./dash
run-detectors: unable to find an interpreter for ./dash
```

You can fix that by registering APE with `binfmt_misc`:

```
sudo wget -O /usr/bin/ape https://cosmo.zip/pub/cosmos/bin/ape-$(uname -m).elf
sudo chmod +x /usr/bin/ape
sudo sh -c "echo ':APE:M::MZqFpD::/usr/bin/ape:' >/proc/sys/fs/binfmt_misc/register"
sudo sh -c "echo ':APE-jart:M::jartsr::/usr/bin/ape:' >/proc/sys/fs/binfmt_misc/register"
```

You should be good now. APE will not only work, it'll launch executables 400µs faster now too. However if things still didn't work out, it's also possible to disable `binfmt_misc` as follows:

```
sudo sh -c 'echo -1 > /proc/sys/fs/binfmt_misc/cli'     # remove Ubuntu's MZ interpreter
sudo sh -c 'echo -1 > /proc/sys/fs/binfmt_misc/status'  # remove ALL binfmt_misc entries
```

WSL

It's normally unsafe to use APE in a WSL environment, because it tries to run MZ executables as WIN32 binaries within the WSL environment. In order to make it safe to use Cosmopolitan software on WSL, run this:

```
sudo sh -c "echo -1 > /proc/sys/fs/binfmt_misc/WSLInterop"
```

Discord Chatroom

The Cosmopolitan development team collaborates on the Redbean Discord server. You're welcome to join us! https://discord.gg/FwAVVu7eJ4

Support Vector

Platform	Min Version	Circa
AMD	K8	2003
Intel	Core	2006
Linux	2.6.18	2007
Windows	8 [1]	2012
Darwin (macOS)	23.1.0+	2023
OpenBSD	7	2021
FreeBSD	13	2020
NetBSD	9.2	2021

[1] See our vista branch for a community supported version of Cosmopolitan that works on Windows Vista and Windows 7.

Special Thanks

Funding for this project is crowdsourced using GitHub Sponsors and Patreon. Your support is what makes this project possible. Thank you! We'd also like to give special thanks to the following groups and individuals:

- Joe Drumgoole
- Rob Figueiredo
- Wasmer

For publicly sponsoring our work at the highest tier.